5.3.4 Solidity hashing functions

Here are three hashing functions provided by Solidity: SHA256, Keccak (also a 256-bit hash function), and RIPEMD160. Recall from section 5.2.1 that RIPEMD160 is used in the generation of a 160-bit account address from the 256-bit public key of an Ethereum account. Keccak was developed for Ethereum based on the SHA3 (secure hash) algorithms. You'll use Keccak as your hash function for Dapp development because it was implemented for the Ethereum blockchain before SHA3 was finalized as a standard.

How can you compute a hash value for a set of data? Here is some simple Solidity code for the Keccak hash function. You can use this function to compute the Keccak hash, as shown in the next listing.

Listing 5.2 Smart contract for hashing (KHash.sol)

```
pragma solidity >=0.4.22 <=0.6.0;

contract Khash {

bytes32 public hashedValue;
function hashMe( uint value1, bytes32 password) public
{
   hashedValue = keccak256(abi.encodePacked(value1, password));
}
}
```

The function `abi.encodedPacked` packs the parameters (any number) and returns byte representation of different types of parameters, and the `keccak256` function computes the hash. You can use this smart contract to compute the Keccak hash values for 20, 30, and so on with the password 0x426526. The only caveat is that you need to input all 32 bytes for the password, as this long hexadecimal number in the latest version of Remix. This representation is to be expected, because we are in the 256-bit realm when dealing with blockchain computing. Here are the 32 bytes of the password with the leading 0x, indicating that the string is hexadecimal:

```
0x4265260000000000000000000000000000000000000000000000000000000000
```